Response to the Editorial Return

exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Antonio De Leon, Raquel Barata, Raquel Prado, and Bruno Sansó

Prepared July 2026

Dear Editors,

We thank the editorial team for the careful return and for the specific guidance on how to make the submission more suitable for review by the *Journal of Statistical Software*. We are submitting the revised material as a new submission, together with this point-by-point response. The revised manuscript, package, and replication materials now put the software contribution first, use a clearer R object and method design, and provide standalone replication materials organized around `code.R`.

The current submission files are:

- Manuscript PDF: `exdqlm-jss.pdf`
- Software source: `exdqlm_1.1.0.tar.gz`
- Replication materials: `exdqlm-jss-replication.tar.gz`
- Point-by-point response: `response-to-editor.pdf`

The software source submitted with this revision is version 1.1.0 of exdqlm. This version is available from CRAN and is distributed under the MIT License, which is GPL-compatible.

Summary of Major Revisions

We reorganized the manuscript to make the JSS contribution explicit: the prior methodology is now separated from the software contribution, with the extended asymmetric Laplace (exAL) family/static exAL regression, the extended dynamic quantile linear model (exDQLM), the original Markov chain Monte Carlo (MCMC) and importance-sampling variational Bayes (ISVB) strategy, transfer-function methodology, and the general nonconjugate variational inference strategy of Wang and Blei identified as methodological foundations. The revised text then states the software contribution and a new package design and implementation section describes the S3 object workflow, fitting engines, exAL distribution utilities, C++ backend paths, diagnostics, forecasting, plotting, and extension points.

We revised the package design for standard R use: fitted dynamic and static objects now have shared superclass families, diagnostic/forecast/ synthesis objects are visible reusable objects,

and standard methods such as `print()`, `summary()`, `plot()`, `predict()`, and `diagnostics()` are used where natural.

We clarified the computational contributions beyond the earlier methodological papers: Laplace–delta variational Bayes (LDVB) is now the main variational route for the nonconjugate scale–skewness block. This is presented as a model-specific adaptation and implementation of the Laplace–delta strategy for nonconjugate models, with legacy ISVB retained for historical comparisons. The revised manuscript also identifies documented exAL density/cumulative distribution/quantile/sampling utilities, continuous ranked probability score (CRPS) and posterior predictive loss criterion (PPLC) computations for predictive comparison, held-out forecast diagnostics, and the static `rhs_ns` implementation with closed-form shrinkage-block MCMC and variational Bayes updates and evidence lower bound (ELBO) contributions documented in the appendices.

We rewrote the replication materials as a standalone JSS archive. The public workflow is now centered on `Rscript code.R`, with documented quick and single-example options. The public path no longer requires a Git checkout or internal author-maintenance modes.

We merged the former supplement into the main manuscript as appendices and streamlined the appendix material to focus on posterior targets, package-specific LDVB and shrinkage-prior blocks, diagnostics, and predictive synthesis needed for software review.

We rebuilt the final replication archive without Git metadata, author audit notes, model-cache `.rds` files, scratch LaTeX files, or nested archives. We tested the archive from a fresh extraction using the submitted package tarball, and the archive includes the standard-profile outputs and logs generated by the full replication workflow.

For cross-reference, the main manuscript changes appear in the revised Introduction and software-comparison discussion (G1), the diagnostics and forecasting section (D9–D10), the new package design and implementation section and workflow table (G3, D6–D10), the revised reader-facing example code and replication discussion (G2, D1–D5), Examples 3 and 4 (D6, D9–D10), the rewritten conclusion (G1), and the streamlined integrated appendices (D11). The corresponding package changes are reflected in the 1.1.0 source tarball, `DESCRIPTION`, `NEWS.md`, `NAMESPACE`, the help pages, and the test suite. The replication-material changes are reflected in the submitted archive README, `code.R`, `code.html`, reproducibility index, and reproducibility protocol.

1 General Editorial Comments

Comment (G1)

The submission should better indicate which parts of the methodology have already been published and what constitutes the specific contribution for Journal of Statistical Software.

Response

The revised manuscript now separates the methodological foundations from the JSS software contribution more explicitly. It identifies Yan, Zheng, and Kottas as the source for the extended asymmetric Laplace (exAL) family and static exAL regression framework, and Barata, Prado, and Sansó as the source for the extended dynamic quantile linear model (exDQLM), the original dynamic MCMC and importance-sampling variational Bayes (ISVB) strategy, and the transfer-function extension. The revision also identifies Wang and Blei as the source of the general Laplace–delta strategy for variational inference in nonconjugate models.

The manuscript then explains that the contribution of the present article is software-centered. The exdqlm package brings these methods together in a public R implementation with a common object system and standard methods for fitting, examining, diagnosing, plotting, and forecasting models. It provides MCMC and LDVB inference for dynamic and static models, selected compiled routines for faster computation, predictive diagnostics, transfer-function models, and synthesis across separately fitted quantiles. The article also provides examples, technical appendices, and complete replication materials.

We also added a new package design and implementation section before the examples. This section describes the package architecture, fitted and returned objects, available methods, computational options, and ways in which the package can be extended.

Comment (G2)

The replication material was split across many files and it was not clear which option should be used to re-run all computations and obtain the article results.

Response

We rewrote the replication materials with `code.R` as the main public replication file. The first-page replication command is now:

```
Rscript code.R.
```

Two optional commands are documented for convenience:

```
Rscript code.R -quick
```

for a short wiring/test pass, and

```
Rscript code.R -example N
```

for rerunning one manuscript example. The previous public distinction between `portable` and `reference` modes was removed from the reader-facing workflow. Internal exact-value checks used by the authors are kept separate from the public command path.

The revised `README.md` describes the contents of the submitted archive, the package installation options, the expected outputs, and the runtime interpretation. The generated `code.html` is produced from `code.R` using `knitr::spin()` and includes `sessionInfo()`.

Comment (G3)

The package implementation needs improvement, in particular the classes and methods system. A section on package design and implementation would facilitate review.

Response

We revised both the package and the manuscript in response. On the package side, dynamic fits now inherit from a common `exdqlmFit` family, static fits from `exalStaticFit`, and post-processing outputs such as diagnostics, forecasts, forecast diagnostics, posterior-predictive synthesis, and static diagnostics have explicit classes. We expanded standard methods for these objects, including informative `print()` and `summary()` methods and `plot()` methods for diagnostic, forecast, synthesis, and coefficient summary displays. We also added `predict()` support for fitted dynamic objects where it naturally wraps forecasting, and `diagnostics()` methods for dynamic fits, dynamic forecast objects, and static fits.

On the manuscript side, we added a package design and implementation section that describes the object families, fitting engines, returned objects, method dispatch, backend controls, and extension points. The examples were updated to show the preferred object-first workflow.

---

2 Detailed Comments

Comment (D1)

The replication README referred to cloning a GitHub repository. JSS replication material should be a standalone directory without requiring unstable external GitHub repositories.

Response

We rewrote the submitted `README.md` as an archive-oriented document. It no longer asks the reader to clone or pull a GitHub repository. It begins by describing the contents of the submitted replication directory and then gives the commands needed to install the submitted package tarball or use the installed package and run the replication script. A CRAN installation is sufficient when the CRAN version matches the submitted version. The final replication archive has also been rebuilt without `.git` metadata, local repository state, author audit notes, model caches, scratch files, or nested archives.

Comment (D2)

The submitted `code.R` launched `run_all.R`, which itself ran the different analysis parts. A master script may launch subparts, but the master script should not itself be launched by another master script.

Response

The revised `code.R` is now the public master script. It sources the manuscript setup and the example scripts directly in manuscript order rather than delegating the workflow to a second master script. Author-maintenance orchestration remains separate from the public reproduction path. The reader-facing script now has visible sections for setup, examples, checks, and session information.

Comment (D3)

The README advertised many different ways to launch the replication material, and the difference between `-mode portable` and `-mode reference` was unclear.

Response

We simplified the public interface. The submitted archive now documents only the full run, the quick run, and the single-example run. The previous `portable`/`reference` terminology is no longer part of the public replication instructions. Exact-value author checks remain part of our internal acceptance workflow, but they are not required for readers or editors to run the submitted replication materials.

Comment (D4)

Embedding the launched code in a `main()` function prevents users from accessing intermediate analysis results. The scripts should help readers adapt the examples to their own analyses.

Response

We removed the whole-workflow `main()` wrapper from the reader-facing `code.R`. The revised script is organized as a direct, commented replication script with small helper functions only where they clarify repeated tasks. The example scripts write the manuscript figures, tables, and logs to documented locations, and their code can be inspected and adapted independently. This makes the replication workflow easier to understand and modify while preserving the tested process used to generate the manuscript results.

Comment (D5)

Running the documented command failed because the submitted directory was not a Git repository and the validation step checked Git remotes and branch metadata.

Response

We split the public replication path from author-side checks. The default submitted `code.R` workflow no longer requires Git metadata, Git remotes, branch freshness, or a GitHub checkout. We verified this by extracting the final replication archive into a fresh directory with no `.git` metadata and running the documented quick command:

```
Rscript code.R -quick.
```

This quick archive smoke test passed. The standard full manuscript workflow generated the outputs, manifests, logs, and manuscript checks included in the submitted archive.

Comment (D6)

The manuscript lacks a section describing the package and its design, including the methods provided, implementation, possible extensions, and variants.

Response

We added a package design and implementation section before the examples. The section describes the principal object families, class inheritance, fitting engines, diagnostic and forecast objects, synthesis objects, plotting and post-processing methods, backend controls, and extension points. It also clarifies why some operations remain named workflow functions while returned objects use standard R methods for printing, summarizing, plotting, and prediction.

Comment (D7)

It would be preferable to have different, more informative print and summary methods for exdqlm objects.

Response

We audited the package object families and revised the standard display methods. Representative fit, diagnostic, forecast, forecast-diagnostic, synthesis, and static-diagnostic objects now have informative `print()` and/or `summary()` output indicating the object type and key dimensions or workflow quantities. The package test suite includes coverage for these standard methods.

Comment (D8)

It is unclear why the package does not make use of class inheritance. Fitted models could share a generic class with common methods and variant-specific additions.

Response

We introduced shared superclass families while preserving existing first-class names for backward compatibility. Dynamic fitted objects now inherit from `exdqlmFit`; static AL/exAL fitted objects inherit from `exalStaticFit`; and reusable post-processing objects inherit from appropriate diagnostic, forecast, synthesis, or static-diagnostic classes. This lets shared methods dispatch on common fit families while preserving variant-specific behavior where needed.

Comment (D9)

It is unclear why the post-processing functions are not methods. This would seem more natural with a suitable class and method system.

Response

We revised the post-processing workflow to make better use of R methods. Operations applied to an existing fitted or forecast object now use methods where appropriate. For example, fitted dynamic objects support `plot()`, `predict()`, and `diagnostics()`; forecast objects can be evaluated with `diagnostics()`; and static fitted objects support `plot()` and `diagnostics()`. These methods return objects that can then be printed, summarized, or plotted.

We retained named functions for operations that combine information from several objects rather than acting on a single object. In particular, `quantileSynthesis()` combines predictive draws from several separately fitted quantile models, so it remains a named function. The revised manuscript explains this distinction and uses the method-based workflow throughout the examples.

Comment (D10)

The diagnostic workflow should return the object in a normal non-invisible way, with plotting handled by a `plot()` method rather than by a plotting argument.

Response

We changed the diagnostic workflow to be object-first. The `diagnostics()` methods return visible diagnostic objects, and the manuscript examples use the stepwise pattern:

```
diag <- diagnostics(fit)
```

followed by `summary(diag)` or `plot(diag)`. This same object-first style is used for forecast diagnostics and static diagnostics where applicable. Compatibility plotting arguments are no longer the primary workflow shown to users.

Comment (D11)

A single PDF file should be submitted as the manuscript. Supplementary material should be included directly as appendices. JSS has no page limits.

Response

We merged the former supplement into the main manuscript as appendices and are submitting one manuscript PDF, `exdqlm-jss.pdf`. The appendix material was also streamlined to focus on posterior targets, package-specific LDVB and shrinkage-prior blocks, diagnostics, and predictive synthesis needed to review the software implementation. The obsolete standalone supplement PDF and separate supplement bibliography artifact were removed from the submission bundle. The appendix source is now included directly at the end of `exdqlm-jss.tex`, so the manuscript source corresponds to the single compiled PDF.

---

3 Validation and Final Submission Checks

All final checks below were run with R 4.6.0.

- `R CMD check -no-manual -run-donttest exdqlm_1.1.0.tar.gz` completed with 0 errors, 0 warnings, and 0 notes.
- The article reproducibility checks completed with 0 errors and 0 warnings.
- The final replication archive audit found no Git metadata, author audit notes, `.rds` caches, obsolete supplement PDF, obsolete supplement bibliography artifact, LaTeX scratch files, or nested archives.
- A quick smoke test from the extracted replication archive passed using the installed package from the submitted package tarball.
- The submitted archive includes the outputs, logs, manifests, and manuscript checks generated by the standard full replication workflow.

We believe these revisions address the editorial return and make the submission easier to inspect, reproduce, and review as a JSS software article.